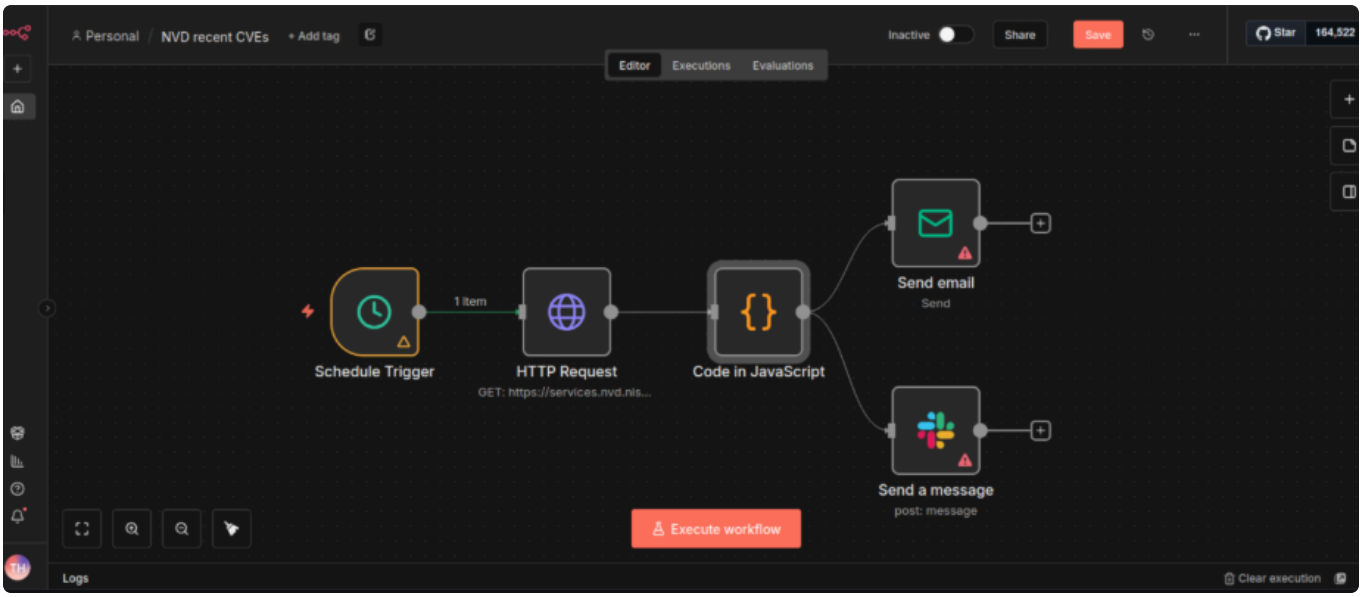
n8n CVE-2025-68613

n8n is an open-source workflow automation platform designed to visually connect applications and services for task automation. Users build workflows composed of nodes, with each node representing an action such as making an API request, processing data, or sending an email. n8n is frequently used to automate repetitive operational tasks and to integrate security tools and SaaS platforms. Below is a simple example workflow that allows us to schedule an HTTP GET request to the NVD CVE API, format the output using JavaScript, and then send the report via email and to a Slack channel.



The n8n platform is commonly deployed in three primary configurations:

- Self-hosted instances: Organizations deploy n8n on-premises or in private cloud environments for full control and data sovereignty
- Cloud-hosted (n8n.cloud): Managed service offering with shared infrastructure
- Internal automation tools: Deployed within corporate networks to automate business processes between internal and external systems

Technical Background

The vulnerability resides in n8n's workflow expression evaluation system, where expressions supplied by authenticated users during workflow configuration are evaluated in an insecure execution context. The core security flaw is an expression injection vulnerability that enables authenticated attackers to execute arbitrary JavaScript code with the privileges of the n8n process. Specifically:

- n8n processes user input wrapped in double curly braces `{{ }}` as JavaScript code

without adequate sandboxing or input validation.
- The expression evaluator lacks proper context isolation, allowing attackers to escape the intended evaluation sandbox.
- Authentication provides no meaningful protection against this vulnerability, as any authenticated user can exploit it.

Consider the following working payload from wioui(opens in new tab)

```
{{ (function(){ return
this.process.mainModule.require('child_process').execSync('id').toString() })()
}}
```

The exploit uses `this.process.mainModule`. Let's break this down:

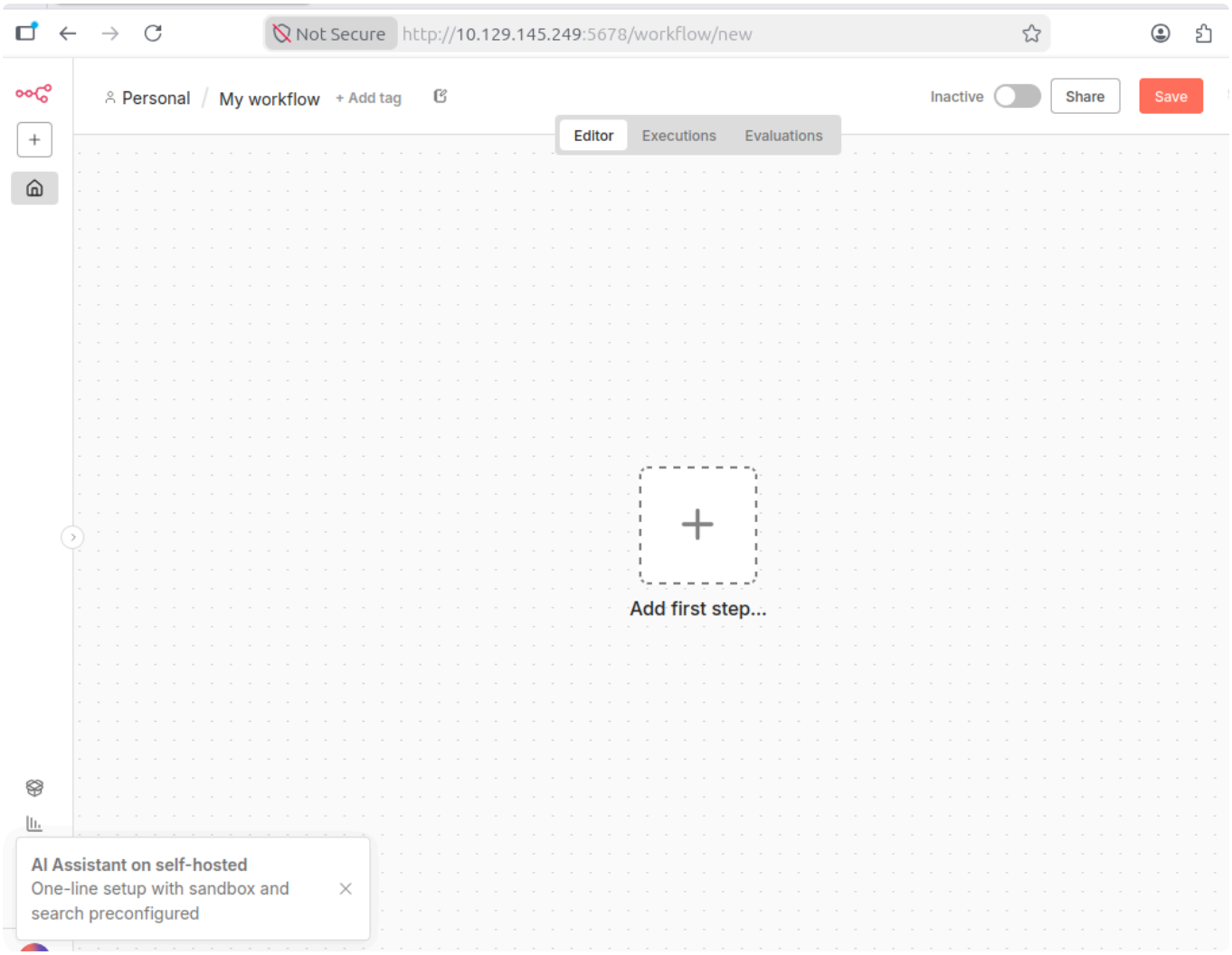
- `this` refers to the global object in the Node.js execution context
- `process` is a Node.js global object providing access to system processes
- `mainModule` references the root module of the Node.js application

the context escalation chain went as follows:

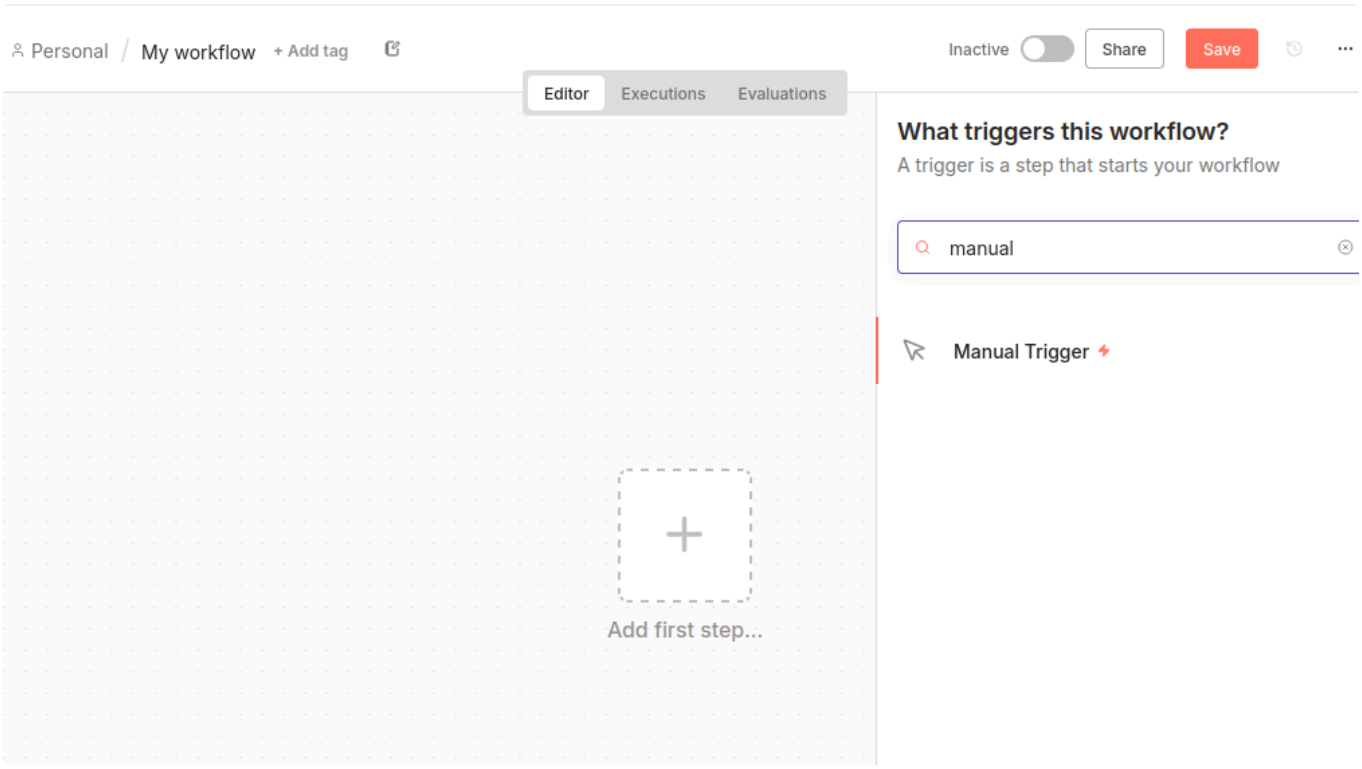
- It starts within the expression evaluator's intended sandbox
- Then it escalates to the Node.js global context via `this`
- Furthermore, it escalates to module system access via `process.mainModule.require`
- Finally, it escalates to system command execution via `child_process`

Exploitation

first i am starting a new workflow from scratch



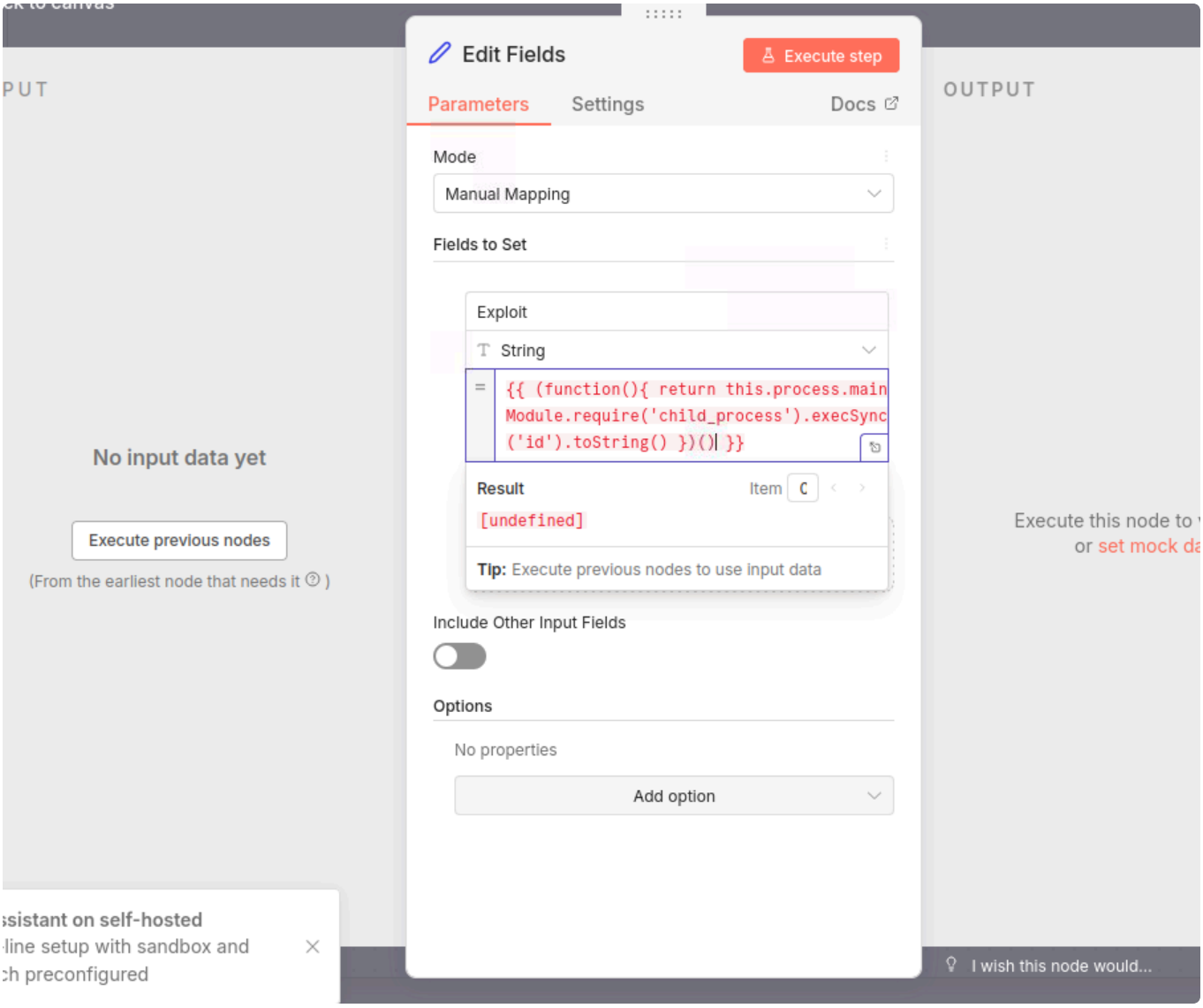
I then have to add a manual trigger first step



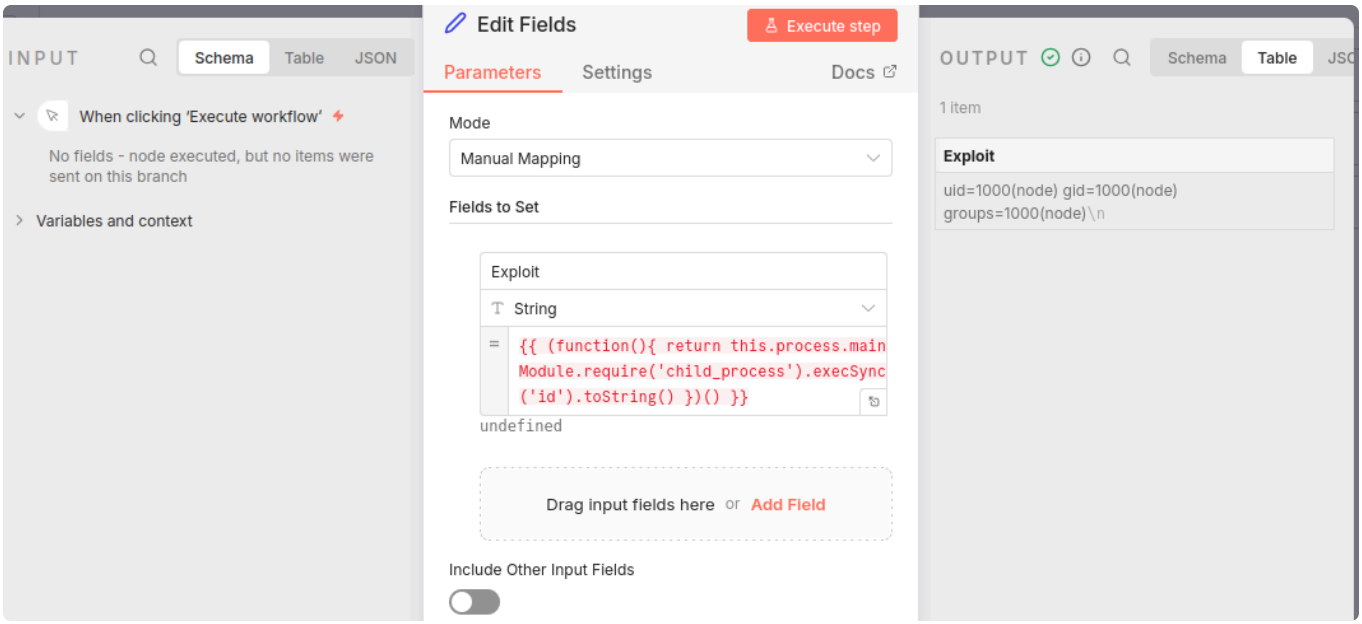
on the manual trigger we add a edit fields (set)

within the edit field I give a name and enter the exploit code within the value section :

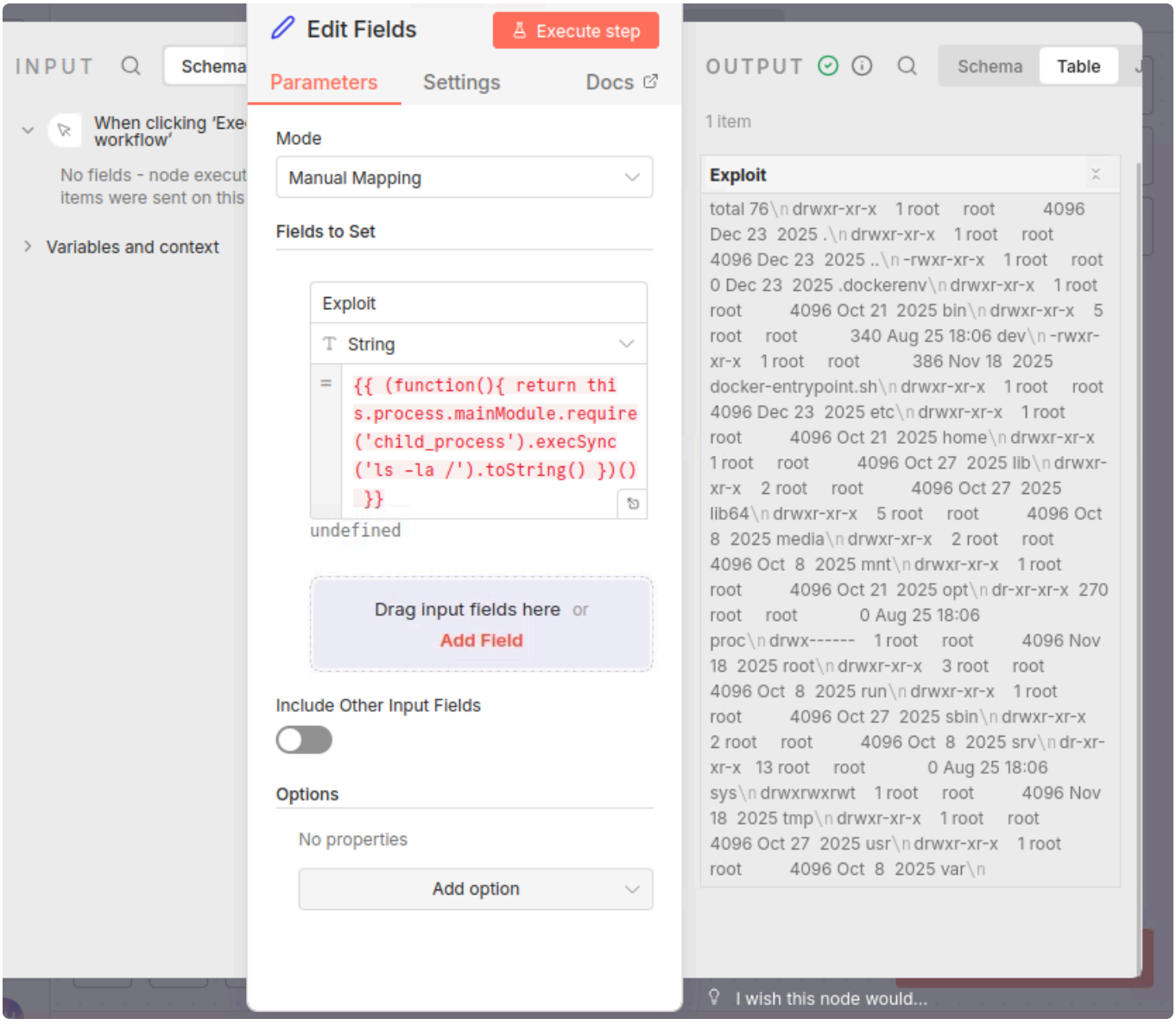
{{ (function(){ return this.process.mainModule.require('child_process').execSync('id').toString() })() }}



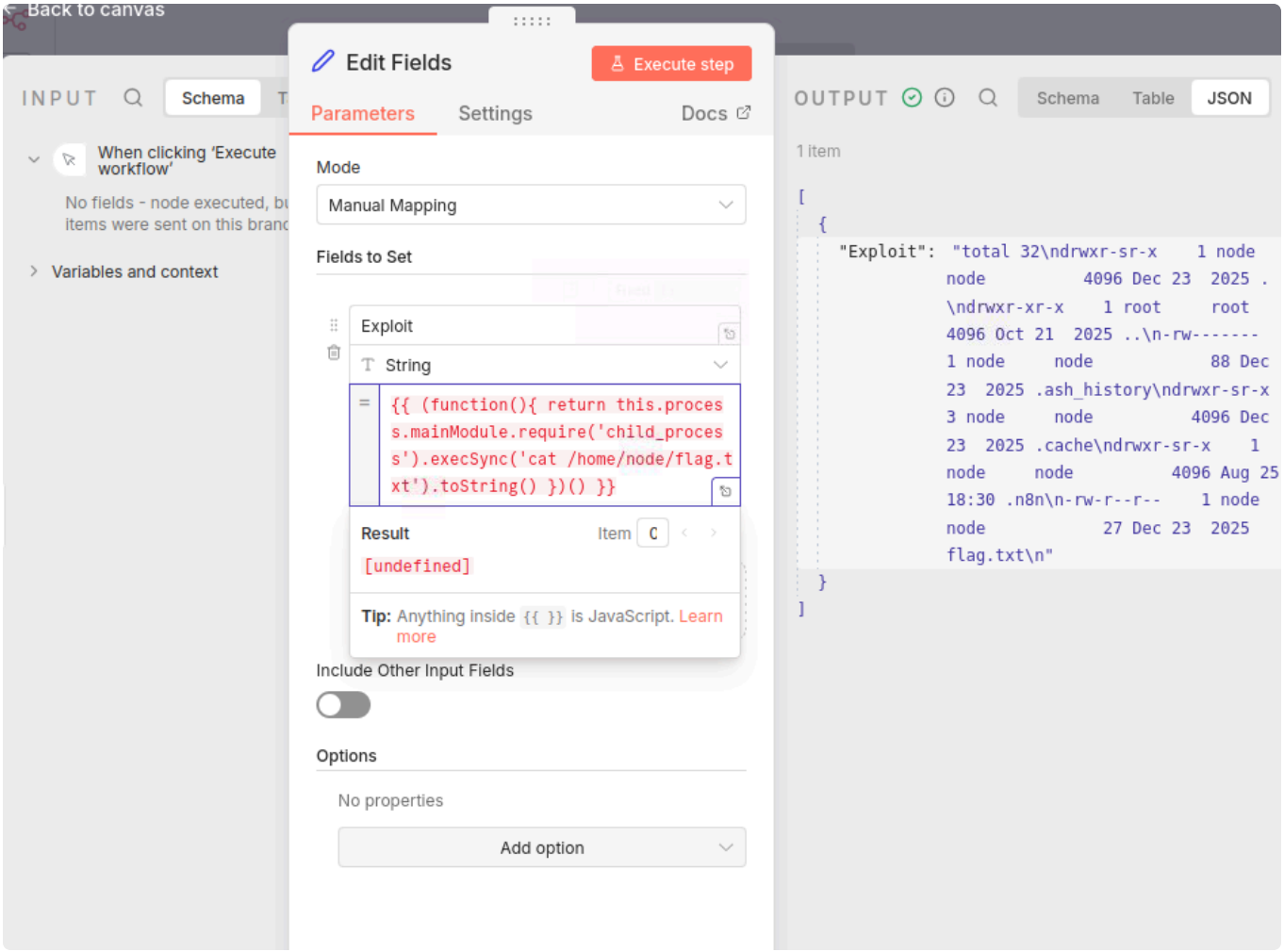
i then executed step and it gave me the ID of a user

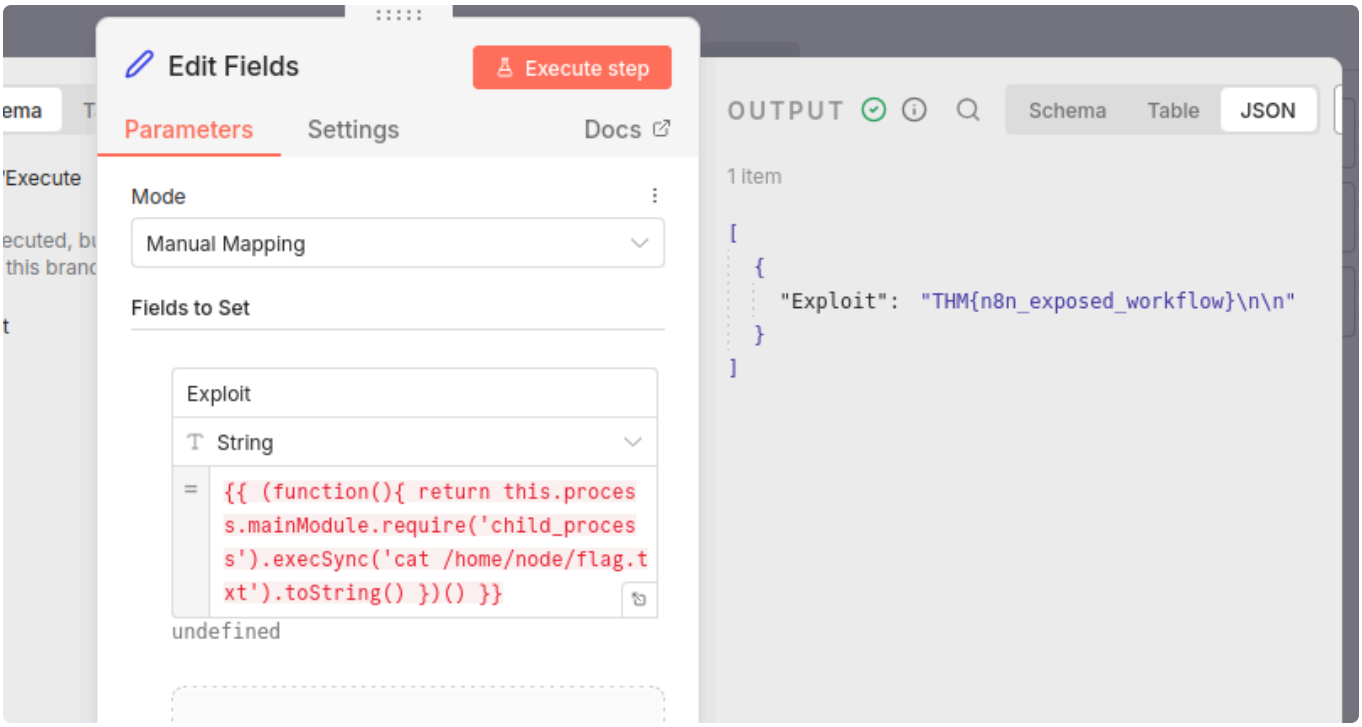


here I can replace ID and inject with other code like i have here listing all files in a directory



i changed the layout to JSON and found the flag.txt file under /node directory and executed it and got the flag





Detection

the best way to detect this attack is by setting up a proxy solution to manage the requests coming to the n8n application. With that approach, you only need to send these proxy logs to your detection solution and then use the body content of the web requests to spot this exploitation.

Below you can see a sample configuration for nginx to log body content with `'Request-Body: "$request_body" '`. Note that this may vary depending on your chosen proxy solution:

```
http {
    # Load Lua module
    lua_package_path "/etc/nginx/lua/?.lua;;";

    # Custom log format
    log_format detailed '$remote_addr - $remote_user [$time_local] '
                        '"$request" $status $body_bytes_sent '
                        '"$http_referer" "$http_user_agent" '
                        'Request-Body: "$request_body" '
                        'Content-Type: "$http_content_type" '
                        'Duration: $request_time s';

    # ... rest of http block ...
```

Sigma Rule

To detect this vulnerability using Sigma, we can use the following rule:

```yaml
title: N8N Workflow RCE Attempt
status: experimental
description: Detects attempts to inject JavaScript expressions into n8n workflow pa
author: TryHackMe Content Engineering Team
references:
  - <https://github.com/wioui/n8n-CVE-2025-68613-exploit>
date: 2025-12-23
tags:
  - attack.execution
  - attack.t1059.007
logsource:
  category: webserver
  product: generic
detection:
  selection:
    cs-method: POST
    cs-uri-stem|endswith: /rest/workflows

  keywords:
    # Strong indicators of this n8n expression injection RCE
    - "this.process.mainModule.require('child_process')"
    - ".execSync("
    - "={{ (function(){"
    - "toString() })()"

  condition: selection and all of keywords
falsepositives:
  - Security testing / red team simulations
  - Developers storing these exact strings in logged fields
level: high
```

Monitoring Suspicious Command Executions

Beyond the previous Sigma rule, it's critical to continue monitoring process creation events to uncover attacker activity after exploitation.

This is critical, since an attacker who only has a valid n8n credentials can fully abuse this RCE to carry out a wide range of malicious actions, such as:

Establishing a reverse shell to gain interactive access to the system.
https://github.com/SigmaHQ/sigma/blob/master/rules/linux/process_creation/proc_creation_lnx_netcat_reverse_shell.yml

Downloading and executing malicious payloads to maintain persistence or escalate impact.
https://github.com/SigmaHQ/sigma/blob/master/rules/windows/process_creation/proc_creation_win_curl_download_direct_ip_exec.yml

Running reconnaissance commands to enumerate the environment hosting the n8n application.
https://github.com/SigmaHQ/sigma/blob/master/rules/windows/process_creation/proc_creation_win_nltest_recon.yml